VERILOG · TEAM PROJECT 02

UART Loopback

Block Diagram · ASM Chart



0x41 'A' · 8N1 · 9600 bps

정안성

Contents

01 **Architecture** p. 2 – 4
uart_loopback Block Diagram · Interface · top_ascii System

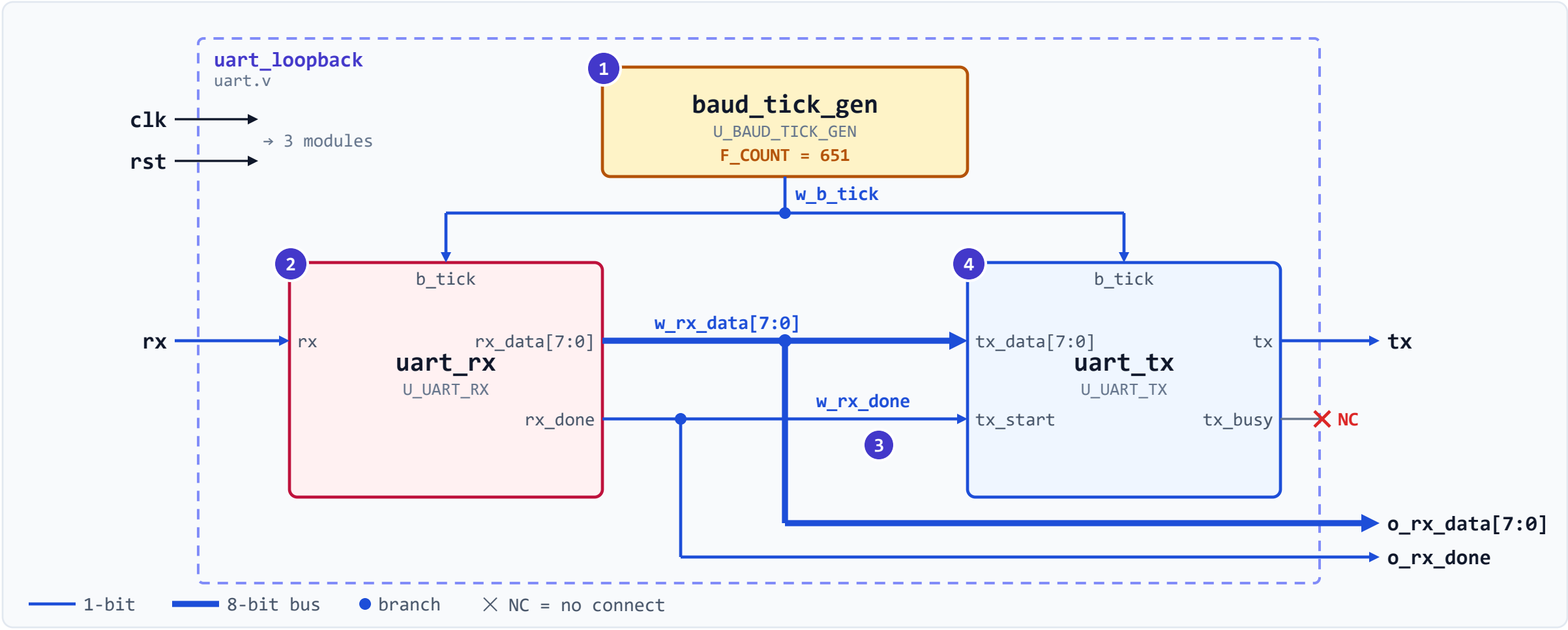
02 **Timing** p. 5
Baud Tick · 16x Oversampling · Bit Center Sampling

03 **ASM Chart** p. 6 – 9
uart_rx ASM · State Table · uart_tx ASM · State Table

04 **Loopback Handshake** p. 10
rx_done → tx_start · Back-to-Back Margin

uart_loopback: RX Output Feeds TX Directly

01



1	**baud_tick_gen**	651 clk마다 1 clk b_tick (9600 × 16)	
2	**uart_rx**	start 중앙 정렬 후 16 tick 간격 sampling	
3	**rx_done → tx_start**	stop bit 중앙 1 clk pulse → echo 시작	
4	**uart_tx**	tx_data latch 후 LSB first 송신	

Interface: rx_done Doubles as tx_start, tx_busy Left Open

01

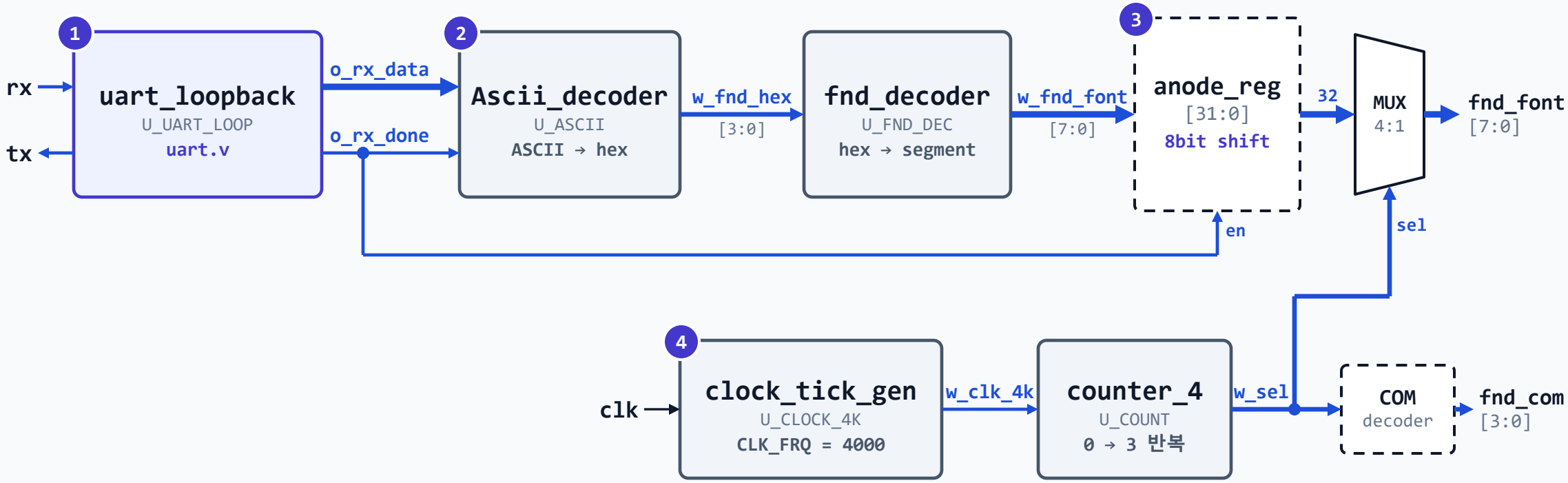
Signal	Width	From → To	역할
clk	1	외부 입력 → 3 modules	100 MHz system clock
rst	1	외부 입력 → 3 modules	active-high, asynchronous reset
rx	1	PC → uart_rx	직렬 입력, idle = 1, synchronizer 없음
w_b_tick	1	baud_tick_gen → uart_rx · uart_tx	6.51 μs 주기 1 clk pulse (153.6 kHz)
w_rx_data	8	uart_rx → uart_tx.tx_data · o_rx_data	수신 byte, rx_done 시점 유효
w_rx_done	1	uart_rx → uart_tx.tx_start · o_rx_done	stop bit 중앙 1 clk pulse = TX 시작 신호
tx	1	uart_tx → PC	echo 직렬 출력, idle = 1
tx_busy	1	uart_tx → 미연결 (NC)	START ~ STOP 구간 1, IDLE 복귀 시 0
o_rx_data	8	uart_loopback → top_ascii	Ascii_decoder 입력
o_rx_done	1	uart_loopback → top_ascii	anode_reg shift enable

Signal 색 = 출발 module (uart_rx · uart_tx · baud_tick_gen · top port)

top_ascii: Each Received Byte Shifts onto the 4-digit FND

01

top_ascii



실선 = module instance · 점선 = top_ascii 내부 always 블록 · clk · rst → 전 module 공통

1	**uart_loopback**	수신 byte echo + o_rx_data · o_rx_done 출력	
2	**Ascii_decoder**	'0'~'9' · 'A'~'F' · 'a'~'f' → hex, 그 외 0	
3	**anode_reg**	rx_done마다 8bit shift → 최근 4글자 표시	
4	**scan**	4 kHz tick → counter_4 → w_sel → digit · font 선택	

02

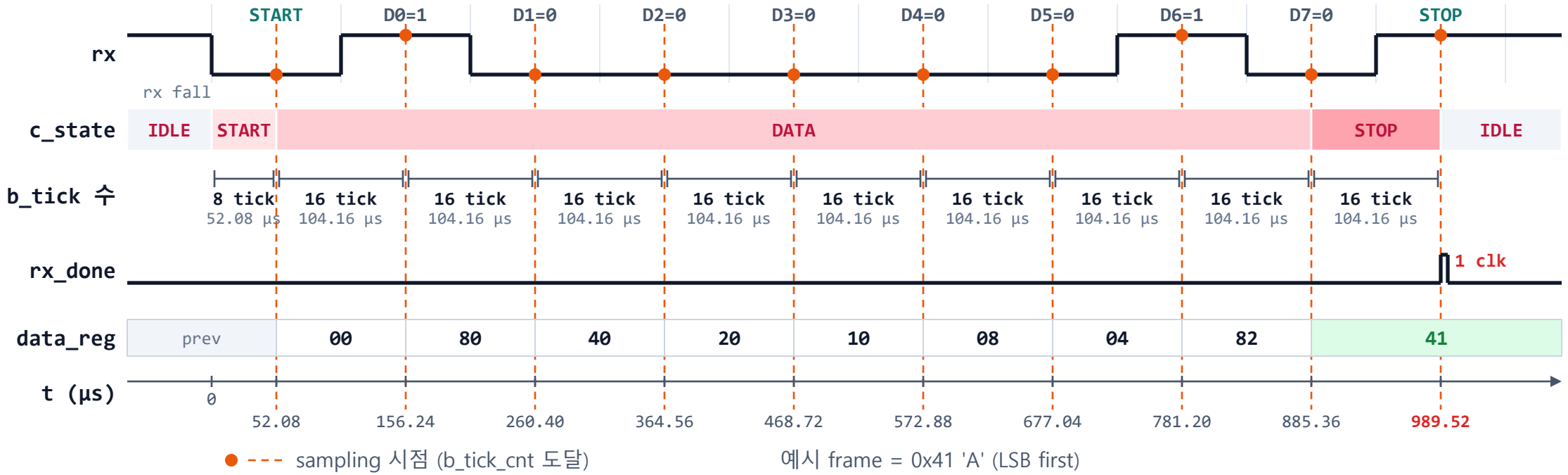
RX Samples Each Bit at Its Center: 8 Ticks, Then Every 16

F_COUNT
100 MHz ÷ 153,600 = **651**

→ **b_tick 주기**
651 × 10 ns = **6.51 µs**

→ **1 bit**
16 tick = **104.16 µs**

→ **start 중앙**
START 8 tick = **52.08 µs**

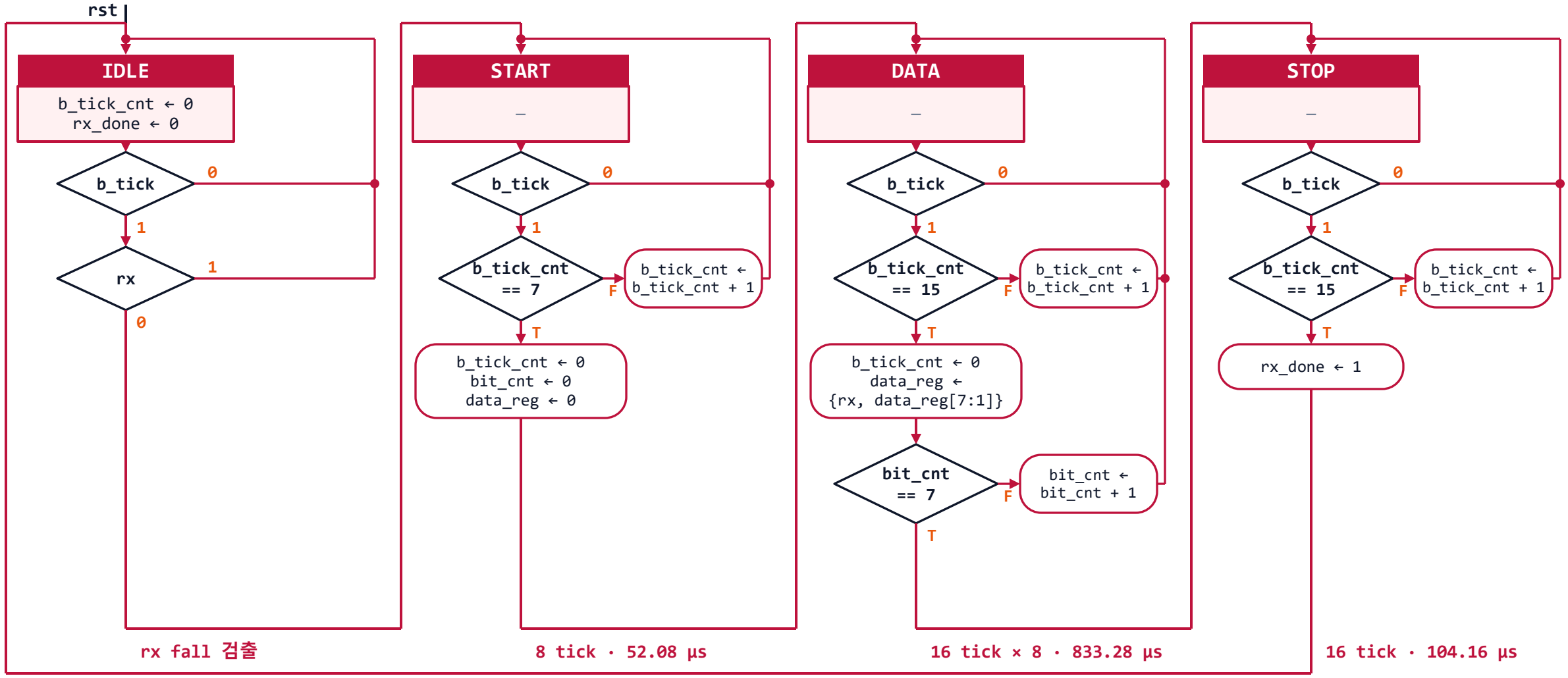


xsim 측정 결과 (tb_measure.sv, 0x41 송신)

b_tick 주기	**6.51 µs**	START 체류	**52.08 µs**	sample 간격	**104.16 µs**
첫 sample	**rx fall + 156.245 µs**	rx_done	**rx fall + 989.525 µs**	data_reg	**0x41 복원**

uart_rx ASM: Align to Start Center, Then Sample Every 16 Ticks

03



State box (무조건 동작) · Decision box · Conditional output box · ● path merge · ← 다음 clock edge 갱신 · T / F 조건 참 / 거짓

uart_rx: 8 + 128 + 16 Ticks from Start Edge to rx_done

03

State	State box 동작	탈출 조건	체류 시간 (xsim 측정)	다음 State · 조건부 동작
IDLE	b_tick_cnt ← 0 rx_done ← 0	b_tick = 1 & rx = 0	rx fall 후 다음 b_tick (최대 1 tick)	START
START	–	b_tick_cnt = 7 (8 tick)	52.08 µs	DATA bit_cnt ← 0 · data_reg ← 0
DATA	–	b_tick_cnt = 15 & bit_cnt = 7	833.28 µs (104.16 × 8)	STOP 매 bit data_reg shift
STOP	–	b_tick_cnt = 15 (16 tick)	104.16 µs	IDLE rx_done ← 1

1 **Start 중앙 정렬** START 8 tick → start bit 중앙, 이후 16 tick 간격 → 모든 bit 중앙 sampling (첫 sample = 1.5 bit 지점)

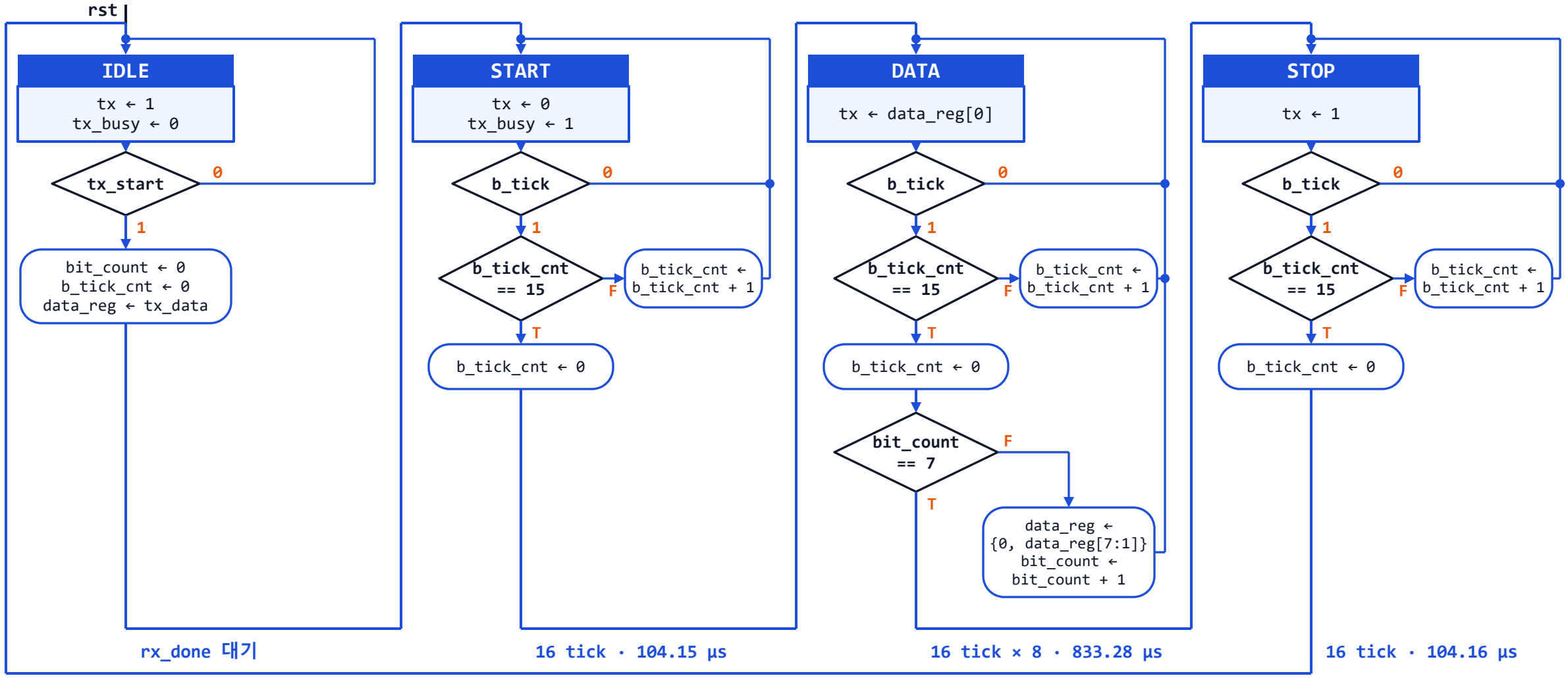
2 **LSB first shift** data_reg ← {rx, data_reg[7:1]} 8회 → 첫 수신 bit(D0)가 bit 0 위치 (0x41 : 80 → 40 → … → 82 → 41)

3 **rx_done 1 clk** STOP 16번째 tick에 1, IDLE 진입 다음 clock에 0 → 10 ns pulse = TX 시작 신호

4 **rx 재검사 없음** START 중앙 rx = 0 · STOP rx = 1 확인 생략 → glitch start · framing error 미검출

uart_tx ASM: Hold Each Bit for 16 Ticks, LSB First

03



State box (무조건 동작) · Decision box · Conditional output box · ● path merge · ← 다음 clock edge 갱신 · T / F 조건 참 / 거짓

uart_tx: tx_start Checked Only in IDLE, Bits Held 16 Ticks

03

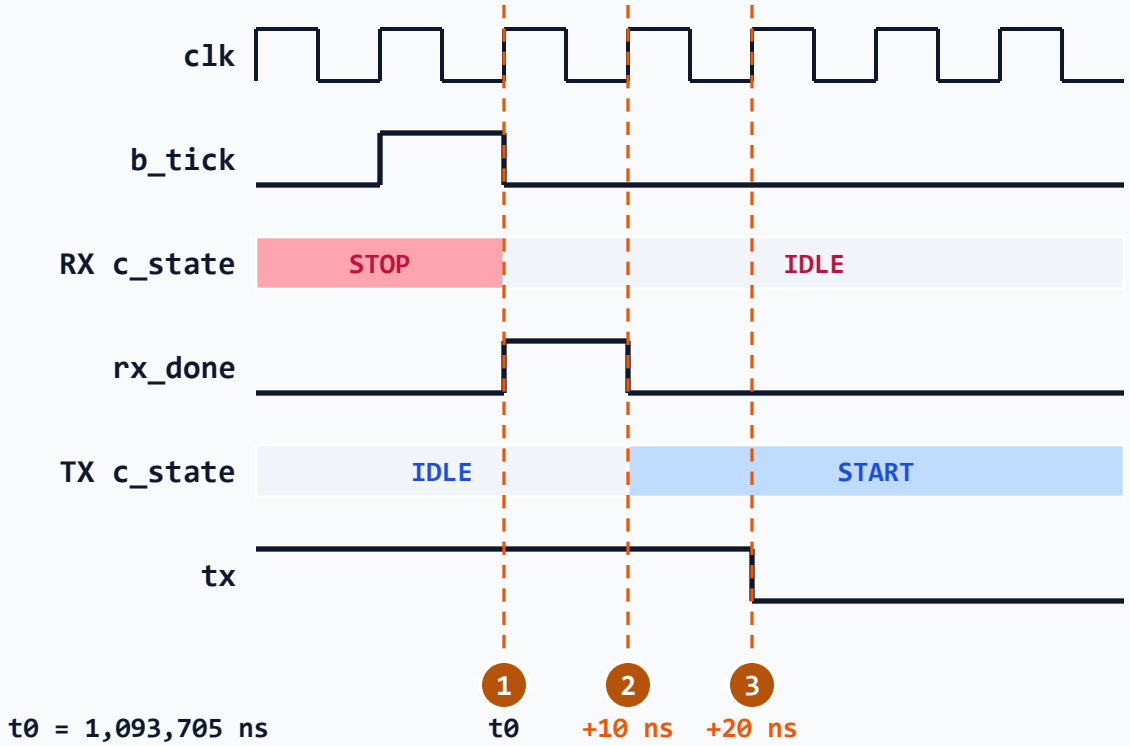
State	State box 동작	탈출 조건	체류 시간 (xsim 측정)	다음 State · 조건부 동작
IDLE	tx ← 1 tx_busy ← 0	tx_start = 1	rx_done까지 대기	START data_reg ← tx_data · 카운터 0
START	tx ← 0 tx_busy ← 1	b_tick_cnt = 15 (16 tick)	104.15 µs	DATA b_tick_cnt ← 0
DATA	tx ← data_reg[0]	b_tick_cnt = 15 & bit_count = 7	833.28 µs (104.16 × 8)	STOP bit마다 data_reg shift
STOP	tx ← 1	b_tick_cnt = 15 (16 tick)	104.16 µs	IDLE

1. **tx_data latch** IDLE에서 tx_start 시점 tx_data → data_reg 복사 → 이후 RX data_reg 변경과 무관
2. **START 104.15 µs** START 진입 = rx_done 다음 clock → 16번째 tick까지 10,415 clk, 이후 bit는 104.16 µs
3. **tx register 출력** tx = tx_reg → state 전환보다 1 clk 늦게 line 변화
4. **tx_busy 미사용** tx_start는 IDLE에서만 검사 → START ~ STOP 중 들어온 rx_done 무시

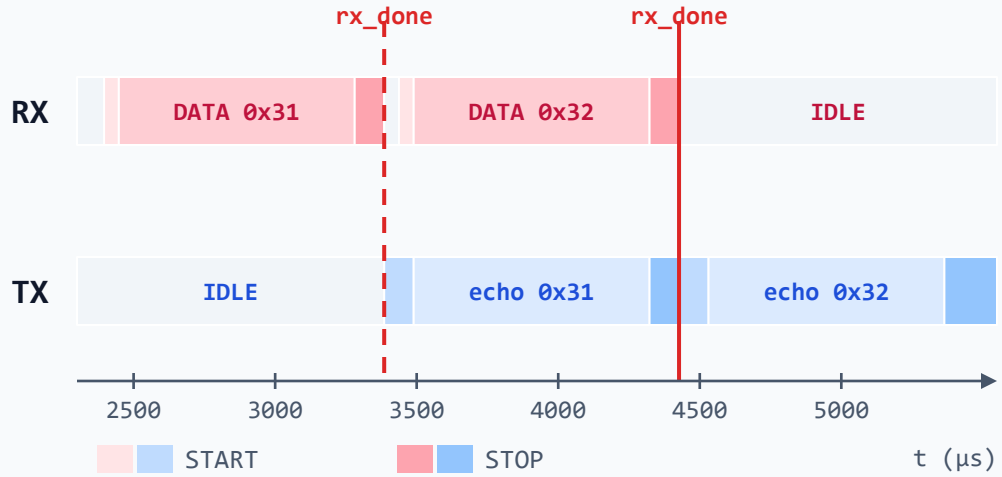
Echo Starts 2 Clocks After rx_done, Back-to-Back Margin 0

04

Clock level · 1st frame (0x41)



Frame level · back-to-back 0x31 → 0x32



4,426,825 ns 동일 clock edge

RX	STOP → IDLE, rx_done ← 1
TX	STOP → IDLE
+10 ns	TX IDLE → START (tx_start 검사 통과)

1. **rx_done 1 clk** RX STOP → IDLE edge에서 1, 다음 edge에서 0 → 10 ns pulse
2. **tx +2 clk** IDLE에서 tx_start 검사 → START +1 clk, tx register 출력 +1 clk → rx_done 후 20 ns
3. **여유 0 tick** TX IDLE 복귀 = 다음 rx_done 동일 edge · rx_done이 1 tick 앞설 경우 TX STOP 중 → tx_start 무시